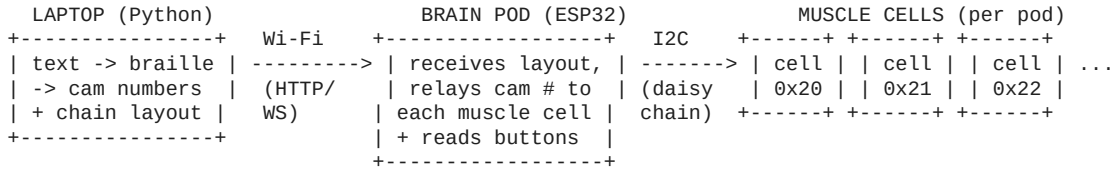
Braillix - Software Team Handoff

> Everything the software team needs to drive the hardware: what runs where, the data pipeline (text -> braille -> motor position), the chain auto-detection rules, and the command protocol. Read this first. Terminology: **Brain Pod** = the ESP32 controller (also called "brain cell"). **Muscle Cell** = one braille character module (motor + cam + 6 dots). One Muscle Cell shows ONE character at a time.

---

1. The 30-second picture



- **Laptop** does the thinking: turns text into braille, into per-cell motor positions,

and decides which cell shows what.

- **Brain Pod (ESP32)** is the messenger + motor coordinator: it talks to the laptop over

Wi-Fi, and to its muscle cells over a 4-wire I2C bus (5V / GND / SDA / SCL).

- **Muscle Cell** just does what it's told: "go to cam position N" -> the motor rotates the

cam so the right combination of the 6 dots pops up.

---

2. What runs WHERE (responsibility split)

Layer	Runs on	Responsibilities
App / conversion	Laptop (Python)	Take user text -> braille cells -> cam numbers (0-63). Build the "chain layout" (who shows what). Send to pods over Wi-Fi. Handle scrolling if text is longer than the display.
Pod firmware	ESP32 (Arduino C++)	Connect to Wi-Fi. Receive commands. **Scan I2C to count its muscle cells.** Forward each cam number to the right cell. Read the 3 nav buttons (Prev/Select/Next) and report to laptop. Mirror status to a web dashboard + serial.
Cell firmware	Muscle-cell board (ATmega, or ULN2003+driver in prototype)	Receive "go to cam position N" over I2C. Home the cam (hall sensor) on power-up. Drive the stepper to that position.

> **Keep the heavy logic on the laptop.** The ESP32 should stay "dumb" - receive numbers, pass them on. This makes the system easy to debug and lets you change braille logic without reflashing hardware.

---

3. The data pipeline (text -> motor) - THIS IS THE CORE

Step 1 - Text to braille cells [OK] ALREADY BUILT, reuse it

`firmware/braille_converter.py` already converts a string into a list of braille cells. Each cell is a tuple of raised dot numbers (1-6). Examples:

- `"a"` -> `[(1,)]`
- `"Hi"` -> `[(6,), (1,2,5), (2,4)]` (capital-indicator, h, i)
- `"1"` -> `[(3,4,5,6), (1,)]` (number-indicator, 1)

```
from braille_converter import convert_text_to_braille
cells = convert_text_to_braille("Hello", skip_unknown=True)
```

Don't rewrite this - it already handles capitals, numbers, punctuation, spaces.

Step 2 - Braille cell to a cam number (0-63)

Each Muscle Cell has a 6-track cam. The 6 dots = 6 bits = a number from 0 to 63. Convert a cell tuple to that number:

```
def cell_to_cam(cell):          # cell = tuple of dots, e.g. (1,2,5)
    value = 0
    for dot in cell:            # dot is 1..6
        value |= 1 << (dot - 1)  # dot1=bit0, dot2=bit1, ... dot6=bit5
    return value                # 0..63
```

Examples: `()` (blank) -> `0`; `(1,)` -> `1`; `(1,2,5)` -> `19`.

> [!] **ONE THING TO LOCK WITH THE HARDWARE TEAM:** the bit order above (dot1 = bit0 ... dot6 = bit5) must match the physical order of the 6 cam tracks. If a printed cell shows the wrong dots, it's almost always this mapping. Confirm against the cam (`braille_cam.scad`) before assuming the math is wrong.

Step 3 - Send cam numbers to the pods

The pod relays each cam number to the matching muscle cell over I2C; the cell's motor moves there. That's the whole chain: **string -> cells -> numbers -> motors.**

---

4. * Auto-detecting the chain (the part you specifically asked about)

The system is modular - any number of muscle cells can be plugged into a brain pod, and there can be more than one brain pod. The software must not assume a fixed size. Two counts matter:

(A) How many muscle cells are on EACH brain pod

On startup (and whenever asked), the pod **scans its I2C bus** to see which cell addresses respond. Muscle cells use addresses **0x20-0x27** (set by solder jumpers on each board).

```
// ESP32 pseudo-code
int cells[8]; int n = 0;
for (int addr = 0x20; addr <= 0x27; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) cells[n++] = addr;  // this cell exists
}
// n = number of muscle cells on THIS pod; report n to the laptop
```

-> The pod reports its cell count (and their addresses, in physical left-to-right order) to the laptop. **This = how many characters that pod can display.**

(B) How many brain pods exist in total

If multiple brain pods are used, the laptop must know the **total number of pods** and each pod's position in the line. Then it can split a long message across the whole display and **send every pod the same overall layout** so they stay in sync.

What "give the same info to every brain pod" means in practice - the laptop sends each pod a small **layout packet**:

```
{
  "total_pods": 3,           // how many brain pods in the whole display
  "this_pod_index": 0,       // which one this is (0 = leftmost)
  "cells_on_this_pod": 4,    // from the I2C scan above
  "full_text": "hello world",// the entire message (same for every pod)
  "my_slice": [19, 5, 12, 12]// the cam numbers THIS pod should show
}
```

Every pod gets the **same** `full_text` **and** `total_pods` (so they all agree on the message and can re-sync), but each acts only on `my_slice`. The laptop is the single brain that computes the slices from the total cell count across all pods.

> **Decision to lock with the team:** how are multiple pods reached over Wi-Fi? Simplest = each pod is its own Wi-Fi device with its own IP, laptop sends to each. (Pod-to-pod I2C across the dock is also possible but adds complexity - recommend laptop-to-each-pod first.)

---

5. Hardware control facts the firmware needs

Thing	Value	Notes
Motor	28BYJ-48 stepper (geared)	Driven via ULN2003 (prototype) or muscle board
Steps per full revolution	**4096** half-steps	NOT 2048. 8 half-steps x 64:1 gearbox

Thing	Value	Notes
Cam positions	**64** (0-63)	One per 6-bit braille combo. **4096 / 64 = 64 steps per position**
Homing	Hall sensor + magnet on cam	Find position 0 on power-up before moving
Stepper library	AccelStepper (HALF4WIRE)	Already working in `breadboard_test.ino`
I2C pins (ESP32)	SDA = GPIO21, SCL = GPIO22	Master = the pod
Muscle cell I2C addresses	0x20-0x27	Set per board by solder jumpers
Nav buttons (ESP32)	Prev=GPIO32, Select=GPIO33, Next=GPIO25	INPUT_PULLUP, active-LOW
Pogo daisy-chain (4 pins)	5V / GND / SDA / SCL	Same on every dock
Power	5V/3A adapter into the pod	Feeds the whole chain

Homing matters: a stepper has no idea where it is at power-up. Each cell must rotate until the hall sensor sees the magnet (that's cam position 0), then count steps from there. The homing logic already exists in `breadboard_test.ino` - reuse its approach.

---

6. Existing code to start from

- `firmware/braille_converter.py` + `braille_mapping.py` - text->braille (Grade 1). **Reuse.**
- `firmware/main.py` - demo/test runner showing how to call the converter.
- `firmware/breadboard_test/breadboard_test.ino` - working single-cell ESP32 firmware:

AccelStepper motor control, hall homing, **Wi-Fi web dashboard, and OTA (wireless code upload)**. This is your starting template for the pod firmware. It currently drives one motor directly; the production version relays over I2C instead.

> The breadboard firmware also has a **web dashboard** (open the ESP32's IP in a browser) and **OTA** so you can push new firmware over Wi-Fi without a cable. Great for development.

---

7. Suggested command protocol (propose / confirm with team)

Laptop -> Pod (over Wi-Fi, HTTP POST or WebSocket - JSON):

```
GET  /chain          -> { "cells": [32,33,34,35], "count": 4 }   // I2C scan result
POST /show           <- { "positions": [19, 5, 12, 12] }          // cam # per cell, left->right
POST /layout         <- { full layout packet from §4(B) }
GET  /buttons        -> { "prev":0, "select":1, "next":0 }        // or push over WebSocket
POST /home           <- {}                                        // re-home all cells
```

Pod -> Muscle Cell (over I2C): send 1 byte = the cam position (0-63) to the cell's address. Optionally a second byte for commands (home, status). Cell replies with status (homing / moving / ready) when the pod reads from it.

---

8. Open decisions for the software team (lock these early)

1. **Cam bit order** (§3 Step 2) - confirm against the physical cam tracks.
2. **Multi-pod transport** (§4B) - laptop-to-each-pod over Wi-Fi (recommended) vs pod-to-pod.
3. **Scrolling** - if text > total cells, scroll how? (timed auto-scroll, or Next button to advance a page). Buttons are already wired for this.
1. **Grade 1 vs Grade 2 braille** - current converter is Grade 1 (letter-for-letter). Fine to start; Grade 2 (contractions) is a later upgrade.
1. **What the nav buttons do** - propose: Prev/Next = scroll pages, Select = repeat/speak.

---

9. Quick start for a software dev

1. `cd firmware && python main.py` - see the braille converter work right now.
2. Read `breadboard_test.ino` - see motor control + Wi-Fi + homing on real hardware.

3. Build the pipeline in §3, the chain detection in §4, the protocol in §7.
4. Coordinate with hardware on the two [!] items: cam bit order, and I2C addresses per board.

```
text  ->  braille cells  ->  cam numbers (0-63)  ->  Wi-Fi to pod  ->  I2C to cell  ->  motor
        (already coded)     (6-bit, §3)                            (count via scan, §4)
```